

Rapport Final

StoryForge

par William Wan & Hsiao-Wen-Paul LO



Date : 28 Juin 2026

Présentation du projet

StoryForge est une application développée en **JavaFX**, destinée à aider les jeunes auteurs à concevoir des univers de fiction. Elle permet à l'utilisateur de gérer la création de personnages sous forme de fiches, tout en offrant des outils pour scénariser et ordonner les différentes scènes avec les différents personnages. L'ensemble des données créées par l'utilisateur est sauvegardé dans une base de données **MySQL**.

Évolution du projet

L'application a été développée progressivement, étape par étape. Chaque étape a introduit une nouvelle fonctionnalité ou une nouvelle contrainte technique :

- **Étape 1 :** première version JavaFX (histoire + personnages, en mémoire).
- **Étape 2 :** ajout de la persistance MySQL.
- **Étape 3 :** restructuration en couches + principes SOLID.
- **Étape 4 :** ajout des scènes (relations One-To-Many et Many-To-Many).
- **Étape 5 :** recherche, filtrage et statistiques.
- **Étape 6 :** tests unitaires et tests d'intégration.

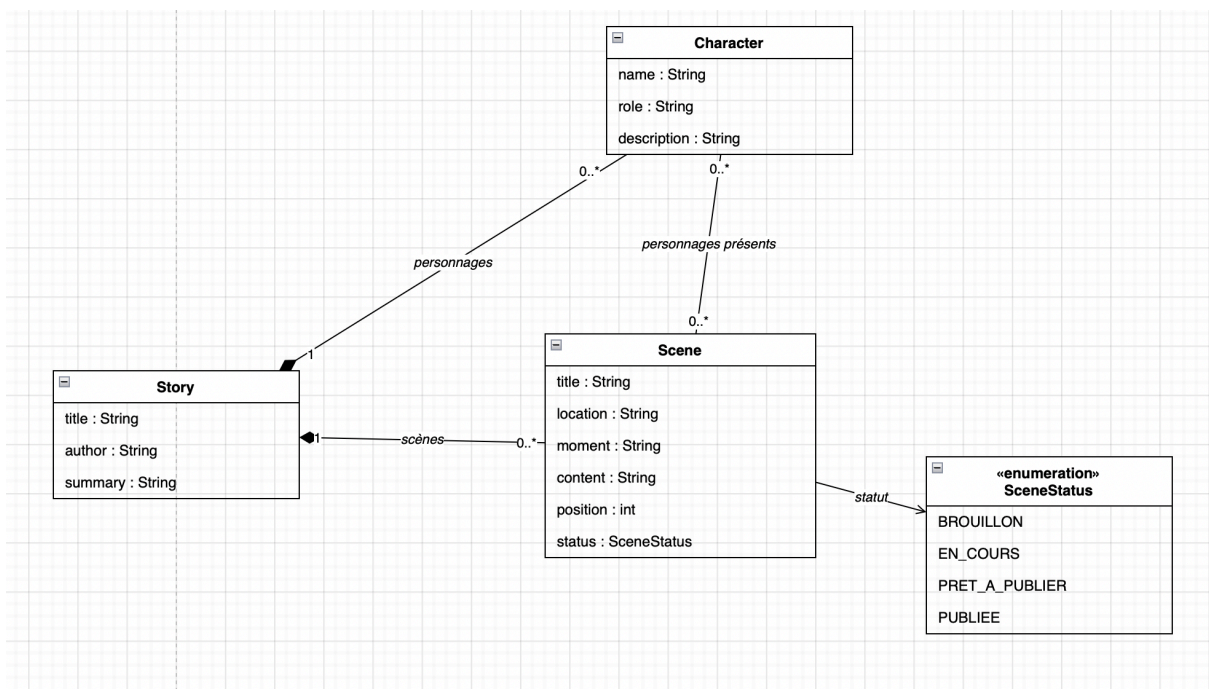
Modèle métier

- Story : une histoire (titre, auteur, résumé).
- Character : un personnage (nom, rôle, description).
- Scene : une scène de l'histoire (titre, lieu, moment, contenu, position, statut).
- SceneStatus : énumération du statut d'avancement d'une scène BROUILLON, EN_COURS, PRET_A_PUBLIER, PUBLIEE.

Les relations entre ces entités sont :

- Story 1 — 0..* Character : une histoire possède plusieurs personnages (composition).
- Story 1 — 0..* Scene : une histoire possède plusieurs scènes (composition).
- Scene 0..* — 0..* Character : une scène contient plusieurs personnages présents, et un personnage peut apparaître dans plusieurs scènes (association plusieurs-à-plusieurs).
- Scene — SceneStatus : chaque scène possède exactement un statut.

Diagramme de classe UML



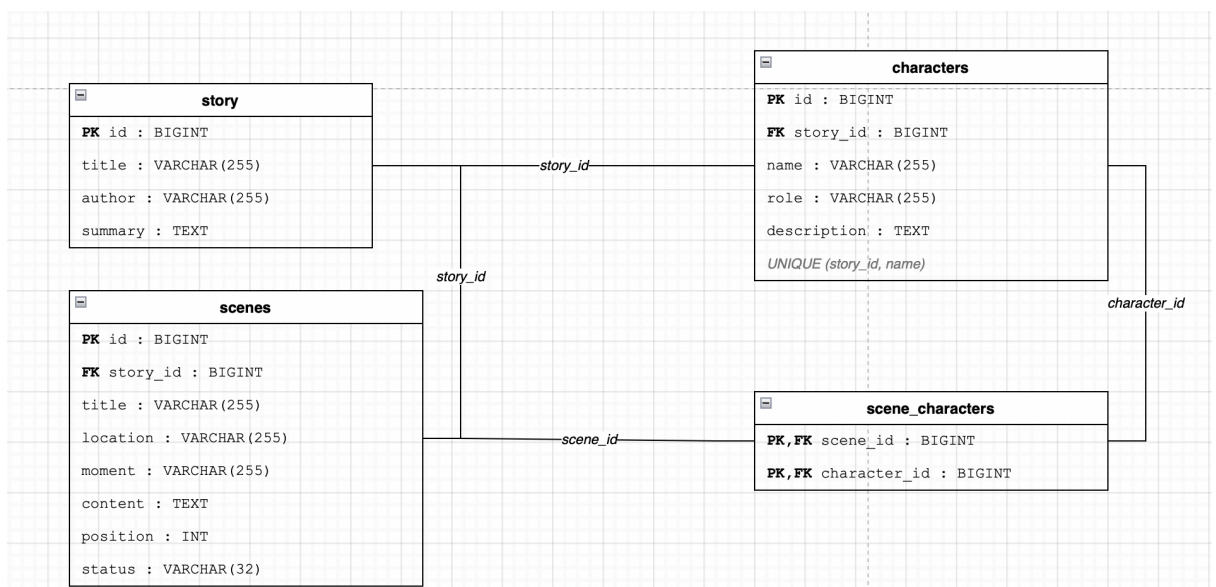
Persistence des données

- Story : id (PK), title, author, summary
- Characters : id (PK), story_id (FK), name, role, description
- Scenes : id (PK), story_id (FK), title, location, moment, content, position, status
- Scenes characters : scene_id (FK), character_id (FK)
-

Relations entre les tables

- story 1 — * characters via la clé étrangère characters.story_id ; story.id (ON DELETE CASCADE).
- story 1 — * scenes via la clé étrangère scenes.story_id ; story.id (ON DELETE CASCADE).
- scenes * — * characters via la table de jonction scene_characters (deux clés étrangères en cascade).

La suppression en cascade est déléguée à MySQL (ON DELETE CASCADE) : supprimer une histoire supprime automatiquement ses personnages, ses scènes et leurs associations.



Comment les objets Java sont sauvegardés

1. Déterminer l'action (Créer ou Modifier) : Le système regarde si l'objet possède déjà un numéro d'identification (id). S'il n'en a pas, c'est qu'il est nouveau : on l'ajoute à la base. S'il en a déjà un, on se contente de mettre à jour ses informations.
2. Récupérer l'identifiant : Lors d'une création, c'est la base de données qui génère ce numéro unique et le renvoie à notre objet Java pour qu'il s'en souvienne.
3. Traduire les données : La base de données ne "parle" pas Java. Il faut donc convertir manuellement les données complexes de l'objet (comme des statuts ou des catégories) en texte simple qu'elle peut comprendre.
4. Éviter les doublons : Quand on relit les données pour recréer les objets en mémoire, le système garde une trace de ce qu'il a déjà chargé pour ne pas créer deux fois le même objet.

Architecture

Le projet est organisé en plusieurs couches afin de bien séparer les responsabilités.

1. Couche présentation (UI)

Cette couche gère l'interface graphique affichée à l'utilisateur.

- Launcher : point d'entrée du programme. Il lance l'application JavaFX.
- StoryForgeApp : charge l'interface FXML et affiche la fenêtre principale.
- MainController : récupère les actions de l'utilisateur (clics sur les boutons, saisie des champs, etc.) et communique avec le service.

2. Couche service

Cette couche contient la logique de l'application.

- StoryService : reçoit les demandes du contrôleur, applique les règles métier et utilise les classes de persistance pour enregistrer ou récupérer les données.

3. Couche persistance (DAO)

Cette couche s'occupe de l'accès à la base de données.

Interfaces DAO :

- StoryRepository
- CharacterRepository
- SceneRepository

Ces interfaces définissent les opérations possibles sur les données (ajout, modification, suppression, recherche).

Implémentations JDBC :

- JdbcStoryRepository
- JdbcCharacterRepository
- JdbcSceneRepository

Ces classes exécutent réellement les requêtes SQL vers MySQL.

- Database : gère la connexion à la base de données MySQL.

4. Modèle métier

Cette couche représente les données manipulées par l'application.

- Story : représente une histoire.
- Character : représente un personnage.
- Scene : représente une scène.
- SceneStatus : représente l'état d'une scène.

Fichier FXML utilisé

main-view.fxml : unique vue de l'application. Elle définit l'interface graphique composée de quatre onglets:

- Connexion MySQL
- Histoires et personnages
- Scènes
- Statistiques
- Les éléments de cette interface sont reliés au contrôleur MainController.

Principes SOLID

Deux principes SOLID sont appliqués dans le projet.

1. Single Responsibility Principle (SRP)

Chaque classe a un rôle précis :

- MainController gère l'interface utilisateur.
- StoryService gère la logique métier.
- JdbcRepository gèrent les accès à la base de données.
- Story, Character et Scene représentent les données de l'application.

Cette séparation rend le code plus clair, plus facile à comprendre et à maintenir.

2. Dependency Inversion Principle (DIP)

Le service (StoryService) utilise les interfaces (StoryRepository, CharacterRepository, SceneRepository) plutôt que les classes JDBC directement.

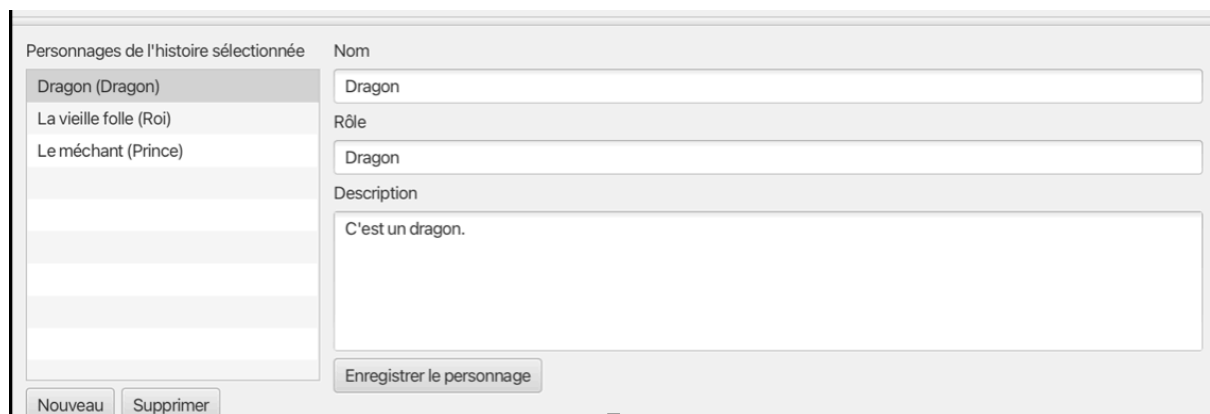
Ainsi, il est possible de remplacer facilement l'accès à la base de données par une autre implémentation (par exemple pour les tests) sans modifier le service.

Captures d'écran



Capture d'écran 1

Gestion des histoires : liste des histoires, formulaire (titre, auteur, résumé) et boutons Nouvelle / Supprimer / Enregistrer.



Capture d'écran 2

Gestion des personnages de l'histoire sélectionnée : casting, formulaire (nom, rôle, description), unicité du nom.

The screenshot shows the StoryForge application window. At the top, there are three tabs: 'Connexion MySQL', 'Histoires et personnages', and 'Scènes'. The 'Scènes' tab is active. On the left, a list titled 'Scènes de l'histoire sélectionnée (gliss...)' contains two items: '1. Le roi et son prince' (highlighted in blue) and '2. Le dragon aux yeux bleus'. The main area on the right is a form for editing a scene. It includes fields for 'Titre' (Le roi et son prince), 'Lieu' (bois de boulogne), and 'Moment' (1). There is a 'Statut' dropdown menu set to 'Publiée'. A large text area for 'Contenu' contains the text: 'Le roi se balade sur son cheval, accompagné du prince, ils discutent de ce qu'ils vont manger à midi.' Below this, a section titled 'Personnages présents (les personnages de l'histoire, sélection multiple)' lists 'Dragon (Dragon)', 'La vieille folle (Roi)', and 'Le méchant (Prince)'. At the bottom left, there are 'Nouvelle' and 'Supprimer' buttons. At the bottom center, there is an 'Enregistrer la scène' button.

Titre	Lieu	Moment	Statut	Contenu	Personnages présents
Le roi et son prince	bois de boulogne	1	Publiée	Le roi se balade sur son cheval, accompagné du prince, ils discutent de ce qu'ils vont manger à midi.	Dragon (Dragon) La vieille folle (Roi) Le méchant (Prince)

Capture d'écran 3

Gestion des scènes : liste ordonnée (glisser-déposer pour réordonner), formulaire (titre, lieu, moment, statut, contenu) et sélection multiple des personnages présents.

Recherche, filtrage et statistiques

Une fois les histoires, les personnages et les scènes enregistrés, l'application permet de les exploiter sans rien ressaisir. Tous ces traitements sont réalisés dans la logique métier (StoryService) et non dans l'interface graphique, comme demandé.

Recherche et filtrage des scènes

La recherche et le filtrage sont intégrés directement dans l'onglet Scènes :

- **Recherche par mot-clé** : seules les scènes dont le titre ou le contenu contient le mot tapé sont affichées (sans tenir compte des majuscules/minuscules).
- **Filtrage par statut** : afficher uniquement les scènes d'un statut donné (Brouillon, En cours, Prêt à publier, Publiée).
- **Filtrage par personnage** : afficher uniquement les scènes dans lesquelles apparaît un personnage choisi.
- Les trois critères se combinent, et un bouton « Réinitialiser » réaffiche toutes les scènes.

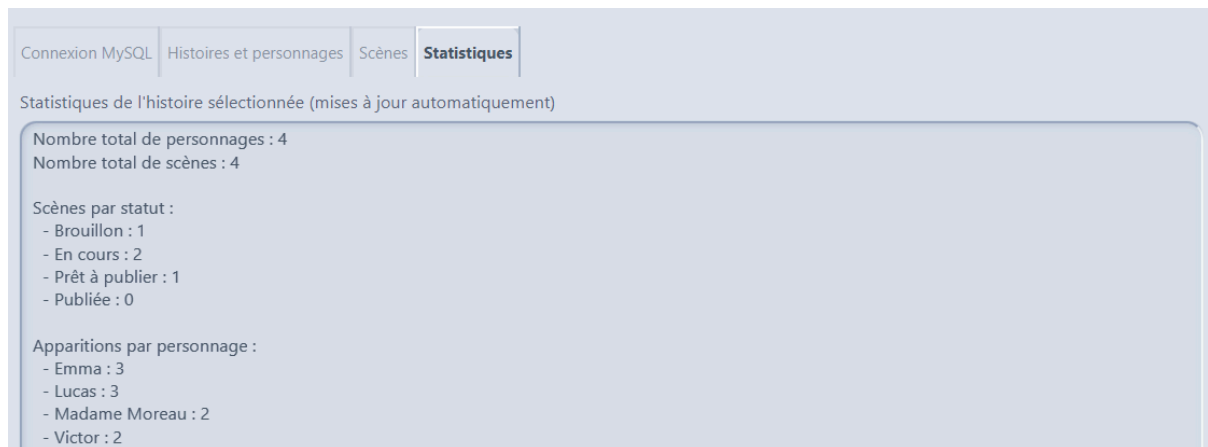
Onglet Scènes : zone de recherche / filtrage (mot-clé, statut, personnage) au-dessus de la liste des scènes.

Statistiques

Un onglet dédié affiche des statistiques sur l'histoire sélectionnée :

- le nombre total de personnages ;
- le nombre total de scènes ;
- le nombre de scènes par statut ;
- le nombre d'apparitions de chaque personnage (sa fréquence dans les scènes).

Ces statistiques sont recalculées automatiquement après chaque modification (ajout, modification, suppression), sans action manuelle de l'utilisateur.



Onglet Statistiques : totaux, répartition par statut et fréquence d'apparition par personnage, mis à jour automatiquement.

Tests réalisés

Pour vérifier automatiquement que les règles importantes fonctionnent, des tests ont été écrits avec JUnit 5. Le plugin maven-surefire-plugin a dû être ajouté au pom.xml pour que la commande « mvn test » exécute réellement ces tests (sans lui, ils étaient silencieusement ignorés).

Tests unitaires (sur le modèle, sans base de données)

- **StoryTest** : refus d'un titre ou d'un auteur vide, unicité du nom d'un personnage dans une histoire, suppression d'un personnage qui le retire aussi de ses scènes, refus d'ajouter à une scène un personnage hors casting, et numérotation des scènes qui reste continue (1, 2, 3...) après suppression ou déplacement.
- **SceneTest** : refus d'un titre ou d'un contenu vide, refus d'un statut nul.

Test d'intégration

StoryServiceTest fait travailler ensemble StoryService, les vraies classes du modèle et trois « fausses » bases de données en mémoire. C'est le principe DIP (le service dépend d'interfaces) qui rend cela possible : on peut tester tout l'enchaînement sans avoir à lancer MySQL. Le test couvre la création d'une histoire / personnage / scène et leur association, ainsi que les recherches, filtrages et statistiques de l'étape 5.

Résultat : 12 tests, 0 échec.

```
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.storyforge:StoryForge >-----
[INFO] Building StoryForge 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ StoryForge ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 3 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ StoryForge ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ StoryForge ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\Users\William\IdeaProjects\StoryForge\src\test\resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:testCompile (default-testCompile) @ StoryForge ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- maven-surefire-plugin:3.2.5:test (default-test) @ StoryForge ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.storyforge.storyforge.model.SceneTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.072 s -- in com.storyforge.storyforge.model.SceneTest
[INFO] Running com.storyforge.storyforge.model.StoryTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.026 s -- in com.storyforge.storyforge.model.StoryTest
[INFO] Running com.storyforge.storyforge.service.StoryServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.042 s -- in com.storyforge.storyforge.service.StoryServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
```

Exécution de « mvn test » : les 12 tests passent (0 échec).

Difficultés rencontrées

La principale difficulté a été la gestion de l'ordre des scènes : après un glisser-déposer ou une suppression, les numéros affichés (« 1. Titre », « 2. Titre », etc.) ne se mettaient pas à jour. Solution : renuméroter automatiquement toutes les scènes à chaque modification pour que les positions restent toujours 1, 2, 3... sans trou.

D'autres difficultés ont été rencontrées au fil des étapes :

- **Démarrer MySQL depuis l'application (Windows)** : la commande « net start » demande les droits administrateur. Solution : la relancer dans une console élevée via PowerShell (Start-Process -Verb RunAs, ce qui déclenche la fenêtre UAC de Windows), puis récupérer son résultat dans un fichier temporaire et le décoder en CP850 (le codage de la console française) pour garder les accents.
- **La contrainte d'unicité de position bloquait le glisser-déposer** : au départ, la base imposait une position unique par histoire. Mais en réordonnant deux scènes, la même position existe brièvement deux fois pendant la mise à jour, ce qui violait la contrainte. Solution : la position est devenue une valeur calculée automatiquement à partir du rang de la scène dans la liste, et la contrainte a été retirée de la base, avec une migration automatique pour les bases déjà créées (elle-même bloquée une fois par l'erreur MySQL 1553, résolue en créant d'abord un index simple sur story_id).

- **Le filtrage cassait le glisser-déposer** : une fois la recherche de l'étape 5 ajoutée, l'ordre affiché ne correspondait plus à l'ordre réel des scènes quand un filtre en masquait certaines. Solution : retrouver la vraie position de la scène avec `getScenes().indexOf(...)` plutôt qu'avec la position de la ligne affichée.

Perspectives d'évolution du projet

- **Chapitres / épisodes** : l'énoncé évoque la possibilité de regrouper les scènes en chapitres. Cette nouvelle entité s'intégrerait dans l'architecture existante sans la remettre en cause.
- **Export** : exporter une histoire (résumé, personnages, scènes) vers un format externe comme le PDF ou le Markdown, pour le partage ou l'impression.
- **Annulation (undo)** : pouvoir annuler une suppression, aujourd'hui irréversible.
- **Multi-utilisateur** : ajouter une gestion de comptes pour permettre une utilisation partagée.

Conclusion

Développée étape par étape, StoryForge est passée d'une simple application en mémoire à un véritable outil de gestion de données narratives : modèle métier riche, persistance MySQL, architecture en couches respectant les principes SOLID, recherche/filtrage/statistiques et tests automatisés. Chaque étape a été l'occasion d'améliorer la structure du code plutôt que d'empiler des fonctionnalités, ce qui a permis d'ajouter facilement de nouvelles entités sans remettre en cause l'existant.